

Introduction to xv6

xv6

- Sixth Edition Unix (UNIX System 6) is written in an obsolete language (pre K&R C) and running on obsolete hardware (PDP-11)
- xv6 is a modern reimplementation of UNIX System 6 (by MIT) in ANSI C for multiprocessor x86 systems.
- Unlike Linux or BSD, xv6 is simple enough to cover in a semester, yet still contains the important concepts and organization of Unix.

xv6

- xv6 compiles using the GNU C compiler, targeted at the x86 using ELF binaries.
 - On BSD and Linux systems, you can use the native compilers;
 - On OS X, which doesn't use ELF binaries, you must use a cross-compiler.
- xv6 does boot on real hardware, but typically we run it using the QEMU emulator.

Learning xv6

- Source code: announced before labs/programming assignments
- xv6 book (link available on the schedule page)
- xv6 source code with line numbers (link available on the schedule page)

QEMU

- QEMU (short for Quick Emulator) is a free and open-source hosted hypervisor that performs hardware virtualization.
 - Hosted (type-2) hypervisor VS. Native/Bare-metal (type 1) hypervisor;
- QEMU emulates CPUs through dynamic binary translation and provides a set of device models, enabling it to run a variety of unmodified guest operating systems.

QEMU

- QEMU can be used together with KVM in order to run virtual machines at near-native speed (requiring hardware virtualization extensions on x86 machines).
- QEMU can also be used purely for CPU emulation for user-level processes, allowing applications compiled for one architecture to be run on another.
- To use QEMU to run xv6, just run “`make qemu-nox`” in the xv6 source directory.

QEMU and GDB

- You can attach GDB to debug the program (even kernel) running in QEMU.
- QEMU has a build-in GDB server that can be attached to outside GDB.
- To use GDB for debugging xv6, boot the OS with “`make qemu-nox-gdb`”

Let's try using the xv6 OS now

- You will need a Linux environment to run the xv6 OS.
 - If you want to use your own laptop, the QEMU needs to be set up.
 - If you are using a CS lab machine, the QEMU tool is already installed.
- Download the base code using the link provided on the course schedule page and unzip it.

Let's try using the xv6 OS now

- cd to the “xv6-demo-BASE” directory (i.e., the root directory of the xv6 OS source code)
- Compile and run the OS by
 - “chmod ugo+x sign.pl”
 - “make qemu-nox”
 - Check the Makefile to see how the target of “qemu-nox” is built, then you will know how the xv6 is built and executed (details next)

Let's try using the xv6 OS now

The current form of the xv6 OS is very rudimentary

- No way to shut down the original xv6 OS.
 - A shutdown command/program was added by the instructor.
- If you use any arrow keys when typing a command, that arrow key input will be part of the command, which causes the OS to fail to find the command.
- ...

Understanding xv6 source code – compilation and launching the OS

- Reading the Makefile is helpful to understand the overall structure of xv6 source code.
- xv6-working-dir\$: **make** **qemu-nox** → target

```
qemu-nox: fs.img xv6.img  
$(QEMU) -nographic $(QEMUOPTS)
```

- Two dependencies of the “qemu-nox” target:
 - fs.img: the file system image
 - xv6.img: the OS image
- Action: `$(QEMU) -nographic $(QEMUOPTS)`
 - QEMU variable (next slide)
 - QEMUOPTS variable:

```
QEMUOPTS = -drive file=fs.img,index=1,media=disk,format=raw \ A  
          -drive file=xv6.img,index=0,media=disk,format=raw \ B  
          -smp $(CPUS) -m 512 $(QEMUEXTRA) C D E
```

A: Specify fs.img as the IDE hard disk 1 of the emulated machine

B: Specify xv6.img as the IDE hard disk 0 of the emulated machine

C: Specify the number of CPU cores as \$(CPUS)

D: Specify the capacity of memory as 512 MB

E: Everything else, currently empty

Understanding xv6 source code – the kernel image

- The OS image (xv6.img)

```
xv6.img: bootblock kernel fs.img
dd if=/dev/zero of=xv6.img count=10000
dd if=bootblock of=xv6.img conv=notrunc
dd if=kernel of=xv6.img seek=1 conv=notrunc
```

“dd” command: copy a file, converting and formatting according to the operands.

```
bootblock: bootasm.S bootmain.c
$(CC) $(CFLAGS) -fno-pic -O -nostdinc -I. -c bootmain.c
$(CC) $(CFLAGS) -fno-pic -nostdinc -I. -c bootasm.S
$(LD) $(LDFLAGS) -N -e start -Ttext 0x7C00 -o bootblock.o bootasm.o bootmain.o
$(OBJDUMP) -S bootblock.o > bootblock.asm
$(OBJCOPY) -S -O binary -j .text bootblock.o bootblock
./sign.pl bootblock
```

```
kernel: $(OBJJS) entry.o entryother initcode kernel.ld
$(LD) $(LDFLAGS) -T kernel.ld -o kernel entry.o $(OBJJS) -b binary initcode entryother
$(OBJDUMP) -S kernel > kernel.asm
$(OBJDUMP) -t kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$$/d' > kernel.sym
```

*OBJS = bio.o console.o exec.file.o fs.o ide.o ioapic.o kalloc.o kbd.o lapic.o
log.o main.o mp.o picirq.o pipe.o proc.o spinlock.o string.o swtch.o syscall.o
sysfile.o sysproc.o timer.o trapasm.o trap.o uart.o vectors.o vm.o*



Core kernel files

Understanding xv6 source code – the user programs (shell commands)

- The file system image (fs.img)

```
fs.img: mkfs README $(UPROGS)
      ./mkfs fs.img README $(UPROGS)
```

```
mkfs: mkfs.c fs.h
      gcc -Werror -Wall -o mkfs mkfs.c
```

```
UPROGS=_cat _echo _forktest _grep _init _kill
      _ln _ls _mkdir _rm _sh _stressfs _usertests
      _wc _zombie
```



User programs in xv6

“ls” at root directory of xv6

```
$ ls
      1 1 512
.      1 1 512
..     1 1 512
README 2 2 1973
cat     2 3 13356
echo    2 4 12572
forktest 2 5 8284
grep    2 6 14992
init    2 7 13124
kill    2 8 12648
ln       2 9 12544
ls       2 10 14768
mkdir   2 11 12696
rm       2 12 12672
sh       2 13 23480
stressfs 2 14 13268
usertests 2 15 58460
wc       2 16 13872
zombie  2 17 12352
```